High-Quality Software Engineering

Part 3: Tools and Rules (Steps 4-5)

Recap

- **Step 1**: quality is our life.
- **Step 2**: the enemy is **complexity**.
- **Step 3**: our answer is **rules** and **tools**.

Part 3: a **closer look** at each.

Step 4

Tools: what we use.

Two Simple Rules for Tools

1. Prefer **high-level** tools.
2. **Automate** repetitive work.

Humans think. Machines repeat.

High-Level Languages

```
total = price * quantity
```

The compiler handles the CPU details.

You focus on the **problem**.

JavaScript

The language of the **web**.

- Runs in every web browser.
- Easy to start: no type declarations.
- **Dynamically typed**: types are checked only **when the program runs**.

JavaScript: The Problem

```
function area(w, h) {
  return w * h;
}
area(10, 5);    // 50
area("10", 5);  // 50 (works by accident)
area("abc", 5); // NaN (no error!)
```

Wrong input gives a **silent wrong result**.

In a big app, users find the bug, not us.

We Need a Higher Level

- JavaScript is **high-level**. It helps us solve problems.
- But it is **not high enough**: nobody checks our types.
- So we need an even **higher-level** language.

What is the solution?

TypeScript

JavaScript **plus types**.

The compiler finds type errors **before** the program runs.

```
function area(w: number, h: number): number {
  return w * h;
}
area("10", 5); // Error caught early!
```

Higher Level for Our Behavior

- A higher-level language lifts our **code**.
- In the same way, our **behavior** must be higher-level too.
- Let machines do repeated work, not humans by hand: that is **automation**.
- **Frameworks** automate much of the coding.

Automated Tests

Write the check **once**. Run it forever.

```
import pytest

def test_guest_price():
    assert price(100, "guest") == pytest.approx(107)
```

Old bugs are caught **immediately**.

Frameworks

Ready-made solutions for common jobs.

- **React**: keeps the screen in sync with data
- **Laravel**: routing, database, login

Less repeated code, fewer mistakes.

Git and CI/CD

- **Git**: no lost or conflicting changes
- **CI/CD**: every change is built and tested **automatically**

```
git commit -m "Add login page"
git push   # tests run automatically
```

The Two Most Important High-Level Tools

Languages and automation lift our **code** and **work**.

The biggest complexity needs high-level **thinking**:

- **Process**: complexity over **time**
- **Design**: complexity in **structure**

Process: Little by Little

Week	Deliver	Learn
1	Guest login	Need SSO
2	SSO login	Need timeout
3	Timeout	Ship!

Small steps, feedback, adjust. (**Agile**)

Design: Rooms and Doors

- **Module** = a room (one job)
- **Interface** = a door (how rooms connect)

Bug in the kitchen? Fix the kitchen.

Design in Code

```
class UserStore:                          # the "door"
    def find_by_email(self, email): ...

class FileUserStore(UserStore): ...   # JSON file
class MySqlUserStore(UserStore): ...  # database

def login(store, email, password):
    user = store.find_by_email(email)
```

File to MySQL: change **one place**.

Design Your Application, Not Just Code

- Don't start by typing code.
- Split the application into **modules**.
- Connect the modules with **interfaces**.

```
login --> UserStore --> FileUserStore
          (interface)   MySqlUserStore
```

Tools Are Not Enough

Agile, CI/CD, React, AI…

You can still ship **slow, unsafe** software.

Tools need **rules**.

Step 4 Summary

- **High-level** and **automation**
- **Process** (time) and **design** (structure)

Step 5

Rules: how we decide.

Key Rules

- **KISS**: Keep It Simple, Stupid
- **DRY**: Don't Repeat Yourself
- **No Surprises**: communicate early
- **Buy, don't build**: reuse

KISS

Choose the **simplest** solution that works.

```
# Complicated
def is_adult(age):
    if age >= 18:
        return True
    else:
        return False

# Simple
def is_adult(age):
    return age >= 18
```

KISS: Warning

"Looks simple" can **hide** complexity.

- Plain-text passwords: simple now, **disaster** later.

```
import bcrypt   # Buy, don't build!
hashed = bcrypt.hashpw(b"secret", bcrypt.gensalt())
```

DRY

Remember the tax example (Part 1)?

```
def price_student(p): return p * 0.9 * 1.07
def price_staff(p):   return p * 0.8 * 1.07
```

Define each fact **once**:

```
TAX = 1.07
def price(p, discount): return p * discount * TAX

def price_student(p): return price(p, 0.9)
def price_staff(p):   return price(p, 0.8)
```

No Surprises

Our #1 **team** rule.

- Tell risks **early**.
- Share changes **early**.
- Update the client and team.

The worst surprise: bad news **on the deadline**.

Buy, Don't Build

Don't write what already exists.

```
# Build: sockets, headers, encoding... by hand
# Buy: one line with a library
import requests
page = requests.get("https://example.com")
```

- Many users already **tested** it.
- Spend your time on the **problem**.

Rules Guide Tools

Rule	Tool use
KISS	Pick the simplest framework
DRY	Functions, not copy-paste
No Surprises	Git, CI, demos
Buy, don't build	Libraries

Step 5 Summary

- Rules = how we **think and decide**.
- Tools do the work. Rules keep us on track.

Finally: Check the Quality

Rules and tools help, but we must still **check** the result.

Step	Login example
Specify	Logged-out users cannot see private pages
Measure	A test tries it on every change
Verify	Result: access denied

Quality is **checked**, not hoped for.

Our Answers

1. **SE**: managing complexity to deliver products under money, time, and people constraints.
2. **High quality**: meets user needs, **easy to change**.
3. **We need**: **rules** (KISS, DRY, No Surprises) and **tools** (languages, tests, Git, frameworks, process, design).